

# 25. 自動校正ロジック関数群

solar\_ws0129-main

2026-07-29

## Contents

|                                           |          |
|-------------------------------------------|----------|
| <b>1 対象</b>                               | <b>2</b> |
| <b>2 このファイルを読む理由</b>                      | <b>2</b> |
| <b>3 呼び出し関係</b>                           | <b>2</b> |
| 3.1 どこから呼ばれるか                             | 2        |
| 3.2 次に読むと理解が進むファイル                        | 2        |
| 3.3 同一リポジトリ内の主要依存                         | 2        |
| <b>4 ソース冒頭の把握</b>                         | <b>2</b> |
| 4.1 代表コード断片                               | 2        |
| <b>5 主要構造の読み取り</b>                        | <b>3</b> |
| 5.1 主要クラス・関数                              | 3        |
| 5.2 ファイルを上から読んだときの定義順                     | 3        |
| 5.3 import 群の詳細                           | 3        |
| <b>6 このファイルを読む前に必要な基礎知識</b>               | <b>3</b> |
| 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する            | 3        |
| 6.2 名前、代入、型、参照                            | 4        |
| 6.3 関数、引数、戻り値、スコープ                        | 4        |
| 6.4 module、package、import、console_scripts | 4        |
| 6.5 例外、try/finally、with、資源解放              | 5        |
| 6.6 制御、状態、入力、モデル予測制御 MPC                  | 5        |
| <b>7 関数・クラスを上から順に解説</b>                   | <b>5</b> |
| 7.1 L8 関数 daytime_stationary_aux_estimate | 5        |
| <b>8 処理の流れ</b>                            | <b>7</b> |
| <b>9 このファイルの位置づけ</b>                      | <b>7</b> |

## 1 対象

| 項目          | 内容                                                               |
|-------------|------------------------------------------------------------------|
| ファイル        | mpc_solarcar/solar_autocal_logic.py                              |
| ソース SHA-256 | 8b67ee6b969cf6d5ee2520b0e3b06ac902a4bbfd00a7684c10d8cc4bb1015529 |
| 種別          | Python                                                           |
| 区分          | runtime helper                                                   |
| 役割          | solar_autocal_node が使う昼間停止時 aux 推定などの純ロジック関数をまとめる。               |
| 起動文脈        | Node から切り出された小さな判定ロジック。                                          |

## 2 このファイルを読む理由

solar\_autocal\_node が使う昼間停止時 aux 推定などの純ロジック関数をまとめる。

- Node 依存を持たないので単体試験しやすい。

## 3 呼び出し関係

### 3.1 どこから呼ばれるか

- mpc\_solarcar/solar\_autocal\_node.py

### 3.2 次に読むと理解が進むファイル

- 特記事項なし。

### 3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

## 4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

Pure helpers for bounded online solar-car calibration.

### 4.1 代表コード断片

```
import math

def daytime_stationary_aux_estimate(
    *,
    ghi_wm2: float,
    day_ghi_threshold_wm2: float,
    speed_kmh: float,
    stationary_speed_kmh: float,
    pack_power_w: float,
    solar_power_w: float,
) -> float | None:
    values = (ghi_wm2, speed_kmh, pack_power_w, solar_power_w)
```

```

if not all(math.isfinite(float(value)) for value in values):
    return None
if float(ghi_wm2) < float(day_ghi_threshold_wm2):
    return None
if abs(float(speed_kmh)) > float(stationary_speed_kmh):
    return None
return max(0.0, float(pack_power_w) + float(solar_power_w))

```

## 5 主要構造の読み取り

主要関数は `daytime_stationary_aux_estimate`。

### 5.1 主要クラス・関数

| 行 | 種別       | 名前                                           | 補足 |
|---|----------|----------------------------------------------|----|
| 8 | function | <code>daytime_stationary_aux_estimate</code> |    |

### 5.2 ファイルを上から読んだときの定義順

| 行 | 内容                                                     |
|---|--------------------------------------------------------|
| 8 | 関数 <code>daytime_stationary_aux_estimate</code> を定義する。 |

### 5.3 import 群の詳細

| 行 | import 文                                             | このファイルで読む理由                                                                  |
|---|------------------------------------------------------|------------------------------------------------------------------------------|
| 3 | <code>from __future__<br/>_import annotations</code> | 型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。 |
| 5 | <code>import math</code>                             | Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L18。      |

## 6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

### 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘\*args’、‘\*\*kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.4 module、package、import、console\_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console\_scripts’ は、端末で使う実行可能名と ‘package.module:function’ を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

## 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

## 6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’ は想定した異常を処理し、‘finally’ は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’ は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’ はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

## 6.6 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を  $x$ 、操作入力を  $u$ 、外乱・予報を  $w$  とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から  $N$  ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

## 根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

## 7 関数・クラスを上から順に解説

### 7.1 L8 関数 daytime\_stationary\_aux\_estimate

- 行範囲: L8–L24
- 所属: module 直下

- 定義: `daytime_stationary_aux_estimate(*, ghi_wm2:float, day_ghi_threshold_wm2:float, speed_kmh:float, stationary_speed_kmh:float, pack_power_w:float, solar_power_w:float)->float|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `all`, `float`, `isfinite`, `max`
- 戻り値の要点: `max(0.0, float(pack_power_w) + float(solar_power_w)) / None / None / None`
- 主なローカル代入名: `value`, `values`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

### 上から順の処理

1. `values` に `(ghi_wm2, speed_kmh, pack_power_w, solar_power_w)` の結果を代入する。
2. 条件 `not all((math.isfinite(float(value)) for value in values))` を判定し、真なら内部処理を行う。
3. `None` を返す。
4. 条件 `float(ghi_wm2) < float(day_ghi_threshold_wm2)` を判定し、真なら内部処理を行う。
5. `None` を返す。
6. 条件 `abs(float(speed_kmh)) > float(stationary_speed_kmh)` を判定し、真なら内部処理を行う。
7. `None` を返す。
8. `max(0.0, float(pack_power_w) + float(solar_power_w))` を返す。

### 代表コード断片

```
def daytime_stationary_aux_estimate(
    *,
    ghi_wm2: float,
    day_ghi_threshold_wm2: float,
    speed_kmh: float,
    stationary_speed_kmh: float,
    pack_power_w: float,
    solar_power_w: float,
) -> float | None:
    values = (ghi_wm2, speed_kmh, pack_power_w, solar_power_w)
    if not all(math.isfinite(float(value)) for value in values):
        return None
    if float(ghi_wm2) < float(day_ghi_threshold_wm2):
        return None
    if abs(float(speed_kmh)) > float(stationary_speed_kmh):
        return None
    return max(0.0, float(pack_power_w) + float(solar_power_w))
```

## 8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

## 9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_autocal_logic.py` を **runtime helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。